Praktikum
Object Oriented Program
2022

EAD Laboratory
EAD



Modul 3

Penanganan Eksepsi dan Multithreading

ASISTEN


ADIS


AFHM


ARTI


CACA


DAPA


DEYA


DIKA


DRFZ


FATH


FDIL


FRQI


HILM


NICA


OLLA


RAIN


SKUY


TITY


WANN


WISH


XASH

EAD Laboratory
EAD

Daftar Isi

Daftar Isi I

Peraturan Praktikum II

Penanganan Exception 1

- Tipe Exception 1
- Pernyataan try dan catch 2
- Mekanisme call stack 2
- Keyword finally 2
- Jenis exception yang sering digunakan 3
- Aturan deklarasi 4

Multithreading 6

- Thread 6
- Interface runnable 7
- Method - method pada thread 8
- Sinkronisasi thread 9

Daftar Pustaka 10

Peraturan Praktikum

1. Setiap peserta praktikum harus datang tepat waktu sesuai dengan jadwal. Toleransi keterlambatan hadir 10 menit. Jika melebihi batas waktu tersebut dan tidak dapat memberikan bukti alasan keterlambatan maka praktikan tidak diperkenankan mengikuti praktikum.

2. Perizinan Praktikum :
    a. Izin berkaitan dengan Sakit atau Kemalangan , maka praktikan dapat memberikan surat perizinan kepada pihak Komisi Disiplin maksimal 3 hari setelah jadwal (shift) praktikum.
    b. Izin lomba atau penugasan institusi tidak berlaku apabila tidak terdapat bukti dispensasi dari Igracias. NB : screenshot dispensasi dari igracias wajib dilampirkan.

3. Seragam Praktikum :
    a. Mahasiswa wajib menggunakan celana bahan hitam (bukan chino atau jeans) pada saat praktikum.
    b. Mahasiswi wajib menggunakan rok hitam/biru gelap panjang tidak ketat pada saat praktikum
    c. Dresscode praktikum (Mengikuti Peraturan Telkom)
        - Senin : Menggunakan kemeja merah telkom atau kemeja putih polos
        - Selasa s/d Rabu : Menggunakan kemeja putih polos
        - Kamis s/d Sabtu : Menggunakan kemeja formal berkerah (bukan kerah sanghai dan bukan polo)
    d. Membuka sepatu saat memasuki ruangan lab.

Penanganan Exception

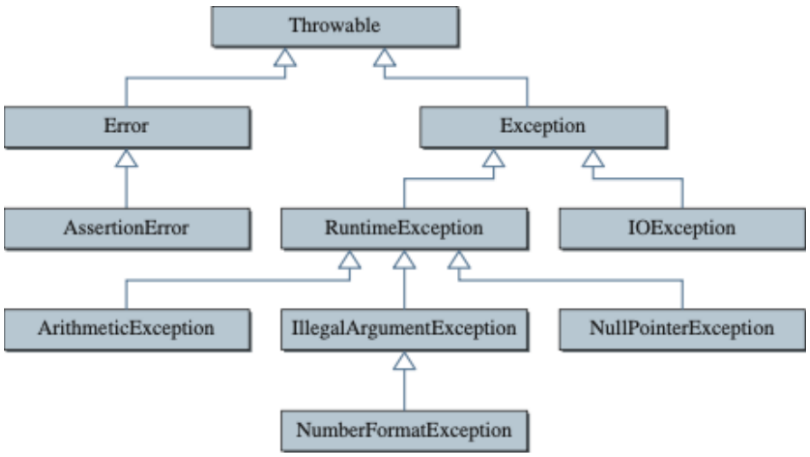
Mekanisme exception handling memungkinkan program untuk menangkap exception, menanganinya, dan meneruskan jalannya program. Struktur exception handling dibuat sedemikian rupa sehingga error dalam program tidak menghalangi jalannya alur normal program. Ketika sebuah error terjadi, ia ditangani dalam blok kode yang terpisah yang diasosiasikan dengan kode eksekusi program yang normal. Hal tersebut akan mempermudah pengaturan kode program.

Tipe Exception

Exception merupakan sebuah kondisi kesalahan yang dapat terjadi selama eksekusi program. Ketika exception terjadi, urutan alur dihentikan dan penanganan exception dijalankan. Ketika exception terjadi, dapat dikatakan exception throw. Ketika kode penanganan eksepsi yang cocok dijalankan, kita katakan exception yang dilempar catch. Dengan menggunakan penanganan pengecualian secara sesuai dalam kode, dapat meningkatkan ketahanannya. Ada 2 tipe exception yang ada di java yaitu :

Checked Exception	Unchecked Exception
Checked exception adalah exception yang muncul akibat dari kondisi eksternal yang dapat saja muncul dari program.	Exception yang terjadi sebagai akibat adanya bug dalam sebuah program atau situasi yang terlalu sulit untuk ditangani.

Adapun class Exception yang disediakan dan bisa digunakan ketika menghandle sebuah throw yang terjadi ketika menjalankan sebuah exception :



Pernyataan try dan catch

Try adalah pernyataan yang menentukan blok kode yang akan diuji kesalahannya saat sedang berjalan. Catch adalah pernyataan yang memungkinkan untuk menentukan blok kode yang akan dieksekusi jika terjadi kesalahan dalam blok kode try.

Untuk menangani suatu exception, tempatkan kode di dalam blok try dan membuat satu atau lebih block catch, di mana masing-masing blok catch diasosiasikan dengan satu jenis exception yang mungkin dilemparkan. Pernyataan di dalam blok catch akan dijalankan jika exception yang terjadi sesuai dengan salah satu exception yang ditangkap oleh salah satu blok catch dalam program.

Mekanisme call stack

Jika sebuah method melemparkan sebuah exception, dan exception ini tidak ditangani di dalam method yang mewadahinya, maka exception tersebut akan dilemparkan ke method yang awal. Jika exception ini juga tidak ditangani saat mencapai metode main (), maka program otomatis berhenti.

Input :

```java
public class App {
    Run | Debug
    public static void main(String[] args) {
        methodPertama ();
    }
    public static void methodPertama (){
        System.out.println(x: "method pertama");
        methodKedua();
    }
    public static void methodKedua (){
        String abc = null;
        abc.toString();
    }
}

```

Output :

```
[Running] cd "d:\OOPKU\OOP3\MODUL3\src\" && javac App.java && java App
method pertama
Exception in thread "main" java.lang.NullPointerException: Cannot invoke "String.toString()" because "<local0>" is null
    at App.methodKedua(App.java:11)
    at App.methodPertama(App.java:7)
    at App.main(App.java:3)

[Done] exited with code=1 in 2.414 seconds
```

Keyword finally

Ketika exception dilemparkan, alur program dalam suatu method membuat jalur yang cenderung tidak linier, melompati baris-baris tertentu, bahkan mungkin akan keluar sebelum waktunya pada kasus di mana tidak ada blok catch yang cocok. Kadang-kadang perlu dipastikan bahwa bagian program yang diberikan akan berjalan, tidak peduli exception apa yang terjadi dan yang ditangkap.

Input :

```java
public class App1 {
    Run | Debug
    public static void main(String[] args) {
    try
    {
        int a = 1/0;
    }
    catch (Exception e)
    {
        System.out.println(x: "Bilangan tidak boleh dibagi 0");
    }
    finally
    {
        System.out.println(x: "Ini untuk keyword finall");
    }
    }
```

Output :

```
[Running] cd "d:\OOPKU\OOP3\MODUL3\src\" && javac App1.java && java App1
Bilangan tidak boleh dibagi 0
Ini untuk keyword finall

[Done] exited with code=0 in 2.253 seconds
```

Jenis exception yang sering digunakan

Terdapat banyak sekali jenis exception yang ada dalam bahasa pemrograman java, namun terdapat beberapa jenis exception yang sering digunakan, diantaranya :

Jenis Exception	Kondisi
ArithmeticException	Ketika terjadi kesalahan dalam perhitungan aritmatik, misalnya membagi angka dengan angka 0
NullPointerException	Ketika berusaha mengakses atribut/method dari sebuah objek menggunakan variabel yang tidak merujuk kepada objek tersebut
NumberFormatException	Ketika mencoba mengubah string menjadi nilai numerik seperti float atau integer, tetapi format string yang ingin diubah tidak sesuai atau ilegal
ArrayIndexOutOfBoundException	Ketika mengakses index array melebihi dari jumlah index yang ada pada array tersebut

IOException	Ketika menginputkan data yang berbeda tipe data (biasanya seharusnya integer/float namun diinputkannya string)
NegativeArraySizeException	Ketika membuat array dengan index negatif

Aturan deklarasi

Untuk memastikan penulisan kode program yang baik, bahasa Java menetapkan bahwa jika sebuah checked exception (yaitu subclass dari kelas Exception tetapi bukan merupakan subclass dari kelas RuntimeException) terjadi di dalam kode program, maka method tempat terjadinya exception harus mendefinisikan secara eksplisit tindakan apa yang harus diambil jika exception muncul. Berikut hal yang harus diperhatikan:

- Tulis blok try() dan catch() pada baris program yang kemungkinan besar akan menghasilkan sebuah exception dan dalam penulisannya harus menuliskan jenis exceptionnya secara khusus atau bisa menuliskan superclass dari exception itu sendiri.
- Membuat method tempat exception terjadi. Dengan cara demikian, exception akan dilemparkan ke method pemanggil.

Selain menggunakan perintah try dan catch, kita juga dapat menggunakan perintah throws dalam menyatakan sebuah exception

void trouble() throws IOException()

Contoh diatas adalah bagaimana cara menyatakan sebuah perintah throws dengan exception yang dinyatakan adalah IOException. Kita juga dapat menuliskan lebih dari satu exception jika memang benar-benar diperlukan. Untuk cara deklarasinya adalah sebagai berikut :

void trouble() throws IOException, ArithmeticException()

Overriding method dan exception

Ketika mengoverride sebuah method yang melemparkan exception, maka method yang mengoverride hanya dapat mendeklarasikan exception yang berasal dari kelas yang sama atau subclass dari exception tersebut. Contoh penerapan method overriding:

Pada contoh berikut ini dideklarasikan tiga kelas, yaitu TestA, TestB1, dan TestB2. TestA merupakan superclass dari kelas TestB1 dan TestB2.



```
40  public class TestA{
41      public void methodA() throws IOException {
42          // do some number crunching
43      }
44  }
45  public class TestB1 extends TestA{
46      public void methodA() throws EOFException {
47          //do some number crunching
48      }
49  }
50  public class TestB2 extends TestA{
51      public void methodA() throws Exception {
52          //do some number crunching
53      }
54  }
55
```

Kelas TestB1 akan berhasil dikompilasi karena EOFException merupakan subclass dari IOException. Tetapi kelas TestB2 tidak berhasil dikompilasi karena Exception merupakan superclass dari IOException.

Multithreading

Dalam satu aplikasi dapat dibuat lebih dari satu thread. Fitur ini dinamakan multithreading. Pada multithreading, thread-thread akan dijalankan “seakan-akan” bersamaan. Karena sebenarnya jika diamati pada low level, sebenarnya thread-thread tersebut tidak berjalan bersamaan, tetapi karena selisih waktu antara eksekusi thread yang satu dengan yang lain terlalu kecil, maka seakan-akan eksekusi thread-thread dirasakan bersamaan oleh manusia.

Thread

Thread adalah sebuah metode pemrograman untuk memungkinkan beroperasi lebih efisien dengan melakukan banyak hal pada waktu yang bersamaan. Tread bisa melakukan pengoperasian di latar belakang tanpa mengganggu program utama. Objek yang datanya akan digunakan dalam thread disebut objek Thread. Objek Thread merupakan instansiasi dari kelas yang merupakan turunan dari kelas Thread, atau ketika kita mengimplementasikan interface Runnable.

Input :

```
8       public Mahasiswa(String n, int s, String a, long d){
9           name = n;
10          semester = s;
11          delay = d;
12          alamat = a;
13          counter = 0;
14      }
15
16      public void run() {
17          while (true) {
18              try {
19                  System.out.println("Nama saya adalah "+ name);
20                  System.out.println("Alamat saya di "+ alamat);
21                  System.out.println("Saya ada di semester "+ semester);
22                  Thread.sleep(delay);
23                  System.out.println(x: "");
24                  counter++;
25                  if (counter ==5){
26                      break;
27                  }
28              } catch (InterruptedException e) {
29                  System.out.println("Something wrong \n desc:"+ e);
30              }
31          }
32      }
        Run | Debug
33      public static void main(String[] args){
34          Mahasiswa mhs1 = new Mahasiswa(n: "John",s: 3,a: "Bandung",d: 2000);
35          mhs1.start();
36      }
37  }
```

Output :



Pada contoh di atas terdapat kelas Mahasiswa yang merupakan turunan dari kelas Thread. Dan pada kelas Mahasiswa juga terdapat method run(). Method ini merupakan overriding method miliki kelas Thread.

Interface runnable

Terdapat masalah ketika kita membuat kelas yang menjadi turunan dari kelas Thread, yaitu tidak dapat diturunkannya kelas tersebut dari kelas lain. Hal tersebut dikarenakan satu kelas hanya dapat merupakan turunan dari satu kelas induk saja. Solusinya adalah dengan mengimplementasikan interface Runnable pada kelas yang akan dibuat.

Runnable merupakan unit abstrak, yaitu kelas yang mengimplementasikan interface ini hanya cukup mengimplementasikan fungsi run(). Dalam mengimplementasi fungsi run(), kita akan mendefinisikan instruksi yang membangun sebuah thread.

Input :

```java
public class MahasiswaTelu implements Runnable {
    private String name;
    private long delay;
    private String alamat;
    private int counter;
    private int semester;

    public MahasiswaTelu(String n, int s, String a, long d){
        name = n;
        semester = s;
        delay = d;
        alamat = a;
        counter = 0;
    }

    public void run() {
        while (true) {
            try {
                System.out.println("Nama saya adalah "+ name);
                System.out.println("Alamat saya di "+ alamat);
                System.out.println("Saya ada di semester "+ semester);
                Thread.sleep(delay);
                System.out.println(x: "");
                counter++;
                if (counter ==2){
                    break;
                }
            } catch (InterruptedException e) {
                System.out.println("Something wrong \n desc:"+ e);
            }
        }
    }
```

```java
                try {
                    System.out.println("Nama saya adalah "+ name);
                    System.out.println("Alamat saya di "+ alamat);
                    System.out.println("Saya ada di semester "+ semester);
                    Thread.sleep(delay);
                    System.out.println(x: "");
                    counter++;
                    if (counter ==2){
                        break;
                    }
                } catch (InterruptedException e) {
                    System.out.println("Something wrong \n desc:"+ e);
                }
            }
        }
        Run | Debug
        public static void main(String[] args){
            MahasiswaTelu mhs1 = new MahasiswaTelu(n: "John",s: 3,a: "Bandung",d: 2000);
            MahasiswaTelu mhs2 = new MahasiswaTelu(n: "Dimas",s: 5,a: "Malang",d: 2000);
            Thread th1 = new Thread(mhs1);
            Thread th2 = new Thread(mhs2);
            try {
                th1.join();
            } catch (Exception e) {
                System.out.println(e);
            }
            th2.start();
        }
    }
```

Pada contoh diatas kita tidak lagi menggunakan Thread sebagai turunan namun diganti dengan melakukan implementasi Runnable pada kelas Mahasiswa. Penerapan Thread dilakukan dengan menginisialisasikan objek baru, objek thread ini membutuhkan parameter berupa objek dari kelas Mahasiswa. Untuk menjalankannya yaitu dengan memanggil method run() pada objek thread yang telah dibuat.

Method - method pada thread

Berikut ini daftar-daftar method yang terdapat pada thread yang digunakan dalam mengeksekusi suatu Thread :

Method	Deskripsi
start()	Memulai/menjalankan proses thread
setPriority()	Method ini mengisi prioritas suatu thread dengan besaran integer yang dimasukkan.
getPriority()	Mengembalikan nilai dari setPriority yang diberikan.
sleep()	Menghentikan proses thread selama waktu yang diberikan pada parameter.
Join()	Menghentikan thread saat ini hingga thread yang dipanggil diakhiri.
currentThread()	Method ini merupakan method statik yang menghasilkan objek Thread yang sedang berjalan.
Yield()	Menyebabkan objek thread yang sedang dieksekusi berhenti sementara dan mengizinkan thread lain untuk mengeksekusi.

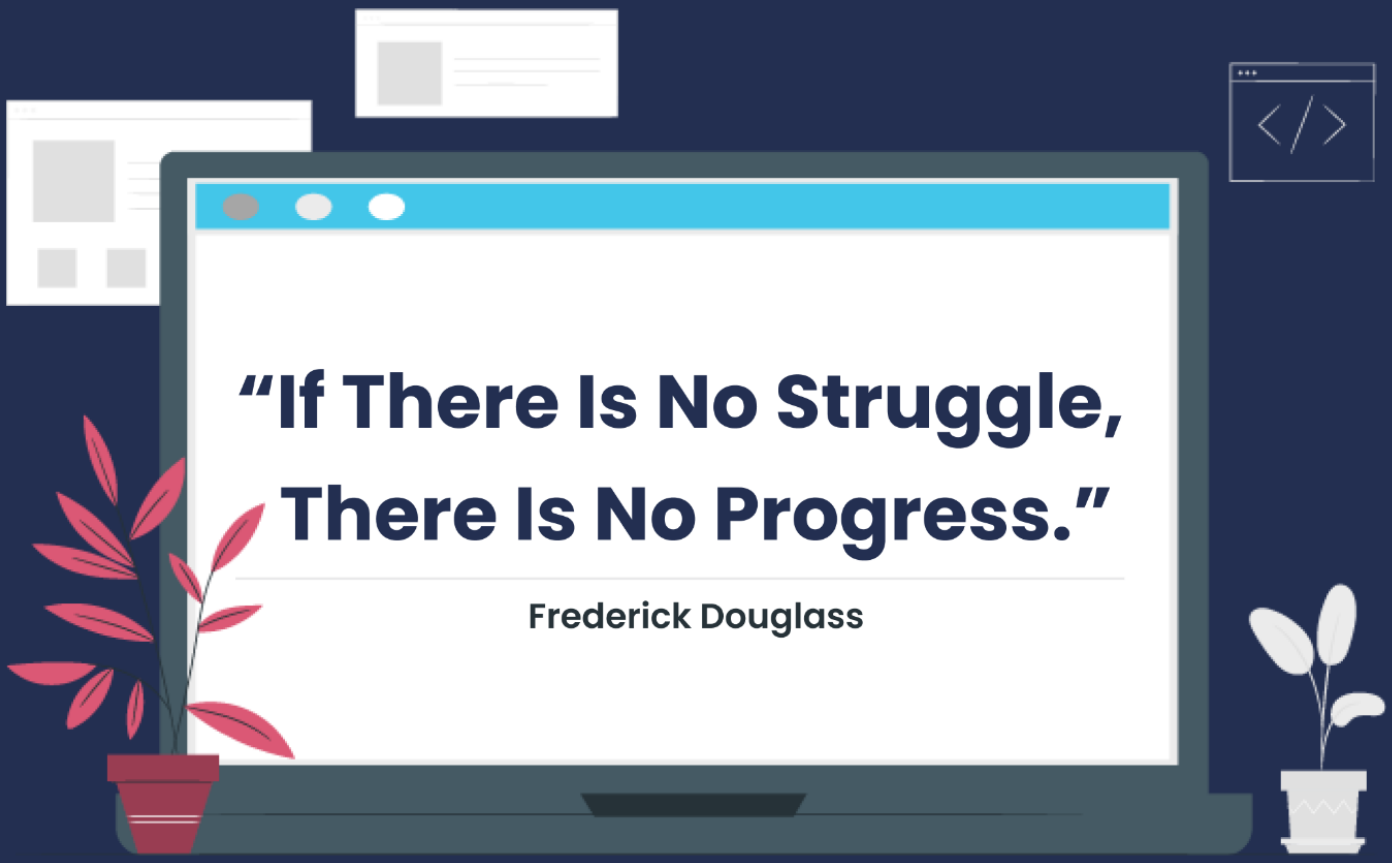
Sinkronisasi thread

Sinkronisasi adalah suatu proses pengendalian akses dari sumber daya terbagi pakai (shared resource) oleh banyak thread sedemikian sehingga hanya satu thread yang dapat mengakses sumber daya tertentu pada satu waktu.

Daftar Pustaka

- Seno Adi Putra, Pemrograman Berorientasi Objek Dengan Teknologi Java, Edisi Pertama, Universitas Telkom, 2019

EAD Laboratory
EAD



Find us on :

@eadlaboratory

@XXX